

# RACER-GT Python API 參考

## 公開工作流程與診斷進入點

Yi-Hao Lai

大葉大學財務金融學系

2026-07-28 • 版本 1.5.0

### 摘要

頂層 API 是穩定的。較低階的函式仍可用於方法論診斷，但演進速度可能快於 pipeline 介面。凡是「拋錯而非回傳降級結果」之處，本文都會明確指出——那些拒絕是刻意的設計決策，不是缺少錯誤處理。

## 目錄

|     |                                                            |   |
|-----|------------------------------------------------------------|---|
| 1   | 頂層 API                                                     | 3 |
| 1.1 | RacerGTConfig                                              | 3 |
| 1.2 | generate_chunk_windows(start, end, window_days, step_days) | 3 |
| 1.3 | generate_collection_schedule(config, anchor_date=None)     | 3 |
| 1.4 | RacerGTPipeline(config).fit(raw, benchmark=None)           | 3 |
| 1.5 | PipelineResult                                             | 4 |
| 2   | 診斷進入點                                                      | 4 |
| 2.1 | 校準                                                         | 4 |
| 2.2 | Duplicates、G-study、consensus                               | 4 |
| 2.3 | Benchmark 與測量誤差                                            | 5 |
| 2.4 | 模擬與驗證                                                      | 5 |
| 3   | 檔案 I/O                                                     | 5 |

## 4 命令列

5

# 1 頂層 API

```
from racergt import (
    RacerGTConfig, RacerGTPipeline, PipelineResult,
    DesignWeightedResult, fit_design_weighted_consensus,
    generate_collection_schedule, generate_chunk_windows,
)
```

## 1.1 RacerGTConfig

巢狀 pydantic 模型，承載所有鎖定設定。所有子模型均使用 `extra="forbid"`，因此 YAML 中出現無法辨識的鍵會是驗證錯誤，而不是被靜默忽略的錯字。

```
cfg = RacerGTConfig.load_yaml("protocol.yaml")
cfg.save_yaml("protocol.lock.yaml")
digest = cfg.protocol_hash()          # canonical 序列化後的 SHA-256
```

`load_yaml` 會重算 hash，若檔案中儲存的 `protocol_hash` 不符即拋錯。手動編輯 lock 檔因此會明確失敗。要改設定，請改來源設定檔後重新產生。

## 1.2 generate\_chunk\_windows(start, end, window\_days, step\_days)

回傳固定的重疊 chunk manifest：chunk\_id、window\_start、window\_end、n\_calendar\_days。

## 1.3 generate\_collection\_schedule(config, anchor\_date=None)

回傳平衡後的 `day × stream × replicate` 排程，含 stream 順序、規劃時段、決定性的 chunk 順序、固定歷史端點與 protocol hash。

**說明 1.1.** 輸出中的 `collection_day` 是序數設計層級 (0、1、2……)，不是 *day offset*；`day_offset` 與 `planned_collection_date` 是另外的欄位。

## 1.4 RacerGTPipeline(config).fit(raw, benchmark=None)

依序執行稽核、各 pull 的重疊校準、duplicate 診斷、設計式參考估計量、G-studies、共變異數調整 consensus、（有提供 benchmark 時的）頻率 benchmark、reliability 診斷，以及鎖定的 decision tree。回傳 PipelineResult。

當原始批次未通過 protocol 稽核時拋出 `ValueError`，除非設定 `stop_on_audit_error=False`

## 1.5 PipelineResult

| 屬性                                 | 內容                                                                    |
|------------------------------------|-----------------------------------------------------------------------|
| <code>final_series</code>          | 最終日頻指標、條件式標準誤、95% 區間                                                  |
| <code>calibrations</code>          | 各 pull 的重疊圖校準結果                                                       |
| <code>duplicate_diagnostics</code> | <code>exact groups</code> 與 <code>near-duplicate</code> 圖             |
| <code>gstudies</code>              | <code>level</code> 、 <code>detection</code> 、 <code>innovation</code> |
| <code>design_consensus</code>      | 設計式參考估計量                                                              |
| <code>consensus</code>             | GLS / 最小變異 consensus                                                  |
| <code>benchmark</code>             | 頻率 benchmark 結果，或 <code>None</code>                                   |
| <code>reliability</code>           | <code>pairwise</code> 、 <code>day/stream</code> 、收斂診斷                 |
| <code>decision</code>              | <code>PASS</code> / <code>REVIEW</code> / <code>FAIL</code> 與每一條規則    |
| <code>audit</code>                 | protocol 完整性稽核                                                        |

`final_series` 在有 `benchmark` 時回傳 `benchmark` 結果，否則回傳 GLS consensus。 `design_consensus` 永遠不會進入它。

## 2 診斷進入點

### 2.1 校準

```
from racergt.overlap import OverlapGraphCalibrator, huber_location,
    OverlapGraphError
```

`OverlapGraphCalibrator.fit` 要求恰好一個 `series_id` 與一個 `pull_id`。當重疊圖不連通且 `allow_disconnected` 為否時拋出 `OverlapGraphError`——因為圖不連通代表相對尺度只被部分識別，此時仍回傳數列等於隱瞞這件事。

### 2.2 Duplicates、G-study、consensus

```
from racergt.duplicates import diagnose_duplicates
from racergt.gstudy import run_gstudy, run_all_gstudies
from racergt.consensus import fit_design_weighted_consensus,
    fit_gls_consensus
```

`run_gstudy` 在設計非完全交叉或 `replicate` 數不等時拋錯，而不是從不平衡設計估出變異成分。`fit_gls_consensus` 要求至少兩個 pull，且每個 pull 的 `baseline` 平均為正且有限。

說明 2.1 (Duplicate 分量不是等價類). *Near-duplicate* 圖的連通分量是連通集合。屬於同一分量不代表其中每一對都超過門檻，因為連通性具遞移性、而 *pairwise* 規則沒有。

## 2.3 Benchmark 與測量誤差

```
from racergt.benchmark import temporal_benchmark
from racergt.eiv import reliability_corrected_ols, simex_ols
```

`reliability_corrected_ols` 在修正後分母非正時拋錯。該情況代表估計的測量誤差已吃掉全部可觀察的解釋變數變異；此時回傳任何數值都毫無意義。

## 2.4 模擬與驗證

```
from racergt.simulation import SimulationSettings, simulate_racergt_data
from racergt.validation import run_monte_carlo
```

`run_monte_carlo` 回傳的 `ValidationResult` 含三個 `frame: metrics` (每個 `replication × 估計量 × 資訊集` 一列)、`summary` (平均值另附 `sd_rmse` 與 `mcse_rmse`) 與 `paired_tests`。每個估計量都在兩個資訊集下各評估一次——僅使用校準後的 `pulls`，以及套用完全相同的 `temporal benchmark`——因為以已 `benchmark` 的 RACER-GT 對比未 `benchmark` 的基準，會把估計量的效果與額外資訊混在一起。`Single pull` 一列取所有 `pull` 的平均而非固定某一欄。`scripts/make_figures.py` 由同一份 CSV 重繪論文圖表。

## 3 檔案 I/O

`racergt.io.read_table` 可讀 CSV、Parquet、XLS/XLSX；`write_table` 可寫 CSV、Parquet、XLSX 與 Stata 118 `.dta`。

## 4 命令列

| 指令                                | 用途                                                        | Exit     |
|-----------------------------------|-----------------------------------------------------------|----------|
| <code>racergt design</code>       | 鎖定 <code>protocol</code> 、排程、 <code>chunk manifest</code> | 0        |
| <code>racergt audit</code>        | 比對批次與鎖定 <code>protocol</code>                             | 失敗為 2    |
| <code>racergt run</code>          | 執行完整 <code>pipeline</code>                                | FAIL 為 3 |
| <code>racergt simulate</code>     | 產生受控資料，可選擇直接執行 <code>pipeline</code>                      | 0        |
| <code>racergt export-stata</code> | CSV 轉 Stata 118                                           | 0        |